Chapter 5 — Joins: Combining Tables

A well-designed database splits information into narrow tables to avoid duplication (chapter 11), so almost any real question — "which department does Neha work in?", "what did Anita buy?" — spans tables. A **join** puts the pieces back together at query time. Joins are SQL's most important skill and its most common interview topic.

Sections: 5.1 Cartesian product and `CROSS JOIN` · 5.2 the four join types side by side (Venn-style pictures — overlapping circles — plus row-by-row results) · 5.3–5.6 `INNER`, `LEFT`, `RIGHT`, `FULL` · 5.7 self joins · 5.8 semi- and anti-joins · 5.9 pitfalls (fan-out, `ON` vs `WHERE`, `NULL` keys, `COUNT(*)`) · 5.10 `USING`, `NATURAL`, multi-table joins and join algorithms.

Running the examples. All queries are in `code/` (`bash tools_run_sql.sh sql '05-*'`), outputs real psql output from PostgreSQL 14 with `NULL` shown as `(null)`. Data: the `company` database (`code/00_schema.sql`, Chapter 1); its foreign keys — every one a join path — are drawn in the chapter 1 ER diagram.

A **foreign key** is a column whose values must match another table's **primary key** (the never-`NULL` column uniquely identifying each row) — `employees.dept_id` must be an existing `departments.dept_id`. Foreign keys are the natural join paths. Quirks: **Asha (CEO) and Rohan (intern) have no department**, **Research** has **no employees**, Hana and Ivan **never ordered**, Desk Lamp was **never sold**, online orders have **no sales rep**.

5.1 Join theory: the Cartesian product and CROSS JOIN

What it is

The **Cartesian product** of two tables (after René Descartes' coordinate pairs) pairs **every row of the first with every row of the second**: $m \times n$ rows. SQL writes it `CROSS JOIN` (or, old style, a comma between tables) — 3 sizes × 2 colours = 6 labels.

```
CREATE TABLE sizes  (size text);
CREATE TABLE colors (color text);
INSERT INTO sizes  VALUES ('S'), ('M'), ('L');
INSERT INTO colors VALUES ('red'), ('blue');

SELECT size, color FROM sizes CROSS JOIN colors ORDER BY size, color;
SELECT count(*) AS pairs FROM sizes, colors;
```

Output:

```
 size | color
------+-------
 L    | blue
 L    | red
 M    | blue
 M    | red
 S    | blue
 S    | red
(6 rows)

 pairs
-------
     6
(1 row)
```

Every join is a filtered Cartesian product

The key idea of the chapter. Conceptually, `A JOIN B ON condition`:

1. Forms every pair (a, b) — the Cartesian product.
2. Keeps only the pairs for which the **join condition** (the `ON` clause) is `TRUE`.
3. For outer joins only: adds back the rows that found **no partner**, padding the missing side with `NULL`s.

(Real databases never build the full product when they can avoid it — §5.10 — but the *result* is defined this way.)

```
SELECT count(*) AS all_pairs FROM employees, departments;
SELECT count(*) AS matching_pairs FROM employees e, departments d WHERE e.dept_id = d.dept_id;
```

Output:

```
 all_pairs
-----------
        90
(1 row)

 matching_pairs
----------------
             13
(1 row)
```

15 × 6 = 90 pairs, only 13 meaningful. Forgetting the join condition — an **accidental Cartesian product** — is a classic bug: the query "works" but returns far too many rows and wildly wrong aggregates.

When CROSS JOIN is what you want

A deliberate cross join builds **every combination**: a report grid (every department × every month, so empty months still appear), test data, a calendar.

```
SELECT d.name, m.month
FROM departments d
CROSS JOIN (VALUES ('2025-01'), ('2025-02')) AS m(month)
WHERE d.dept_id <= 2
ORDER BY d.name, m.month;
```

Output:

```
    name     |  month
-------------+---------
 Engineering | 2025-01
 Engineering | 2025-02
 Sales       | 2025-01
 Sales       | 2025-02
(4 rows)
```

Join syntax: explicit vs implicit

Style	Example	Notes
Explicit (ANSI SQL-92, the 1992 SQL standard)	`FROM a JOIN b ON a.k = b.k`	Join condition sits next to the join; required for outer joins. **Use this.**
Implicit (comma)	`FROM a, b WHERE a.k = b.k`	Older style; easy to forget a condition in a long `WHERE`; can't express outer joins in standard SQL.

Vocabulary: an **equi-join** uses only `=`; a **non-equi join** uses other operators (`<`, `BETWEEN` …); the **join key** is the column(s) compared; **left/right** table mean written before/after `JOIN`.

5.2 The four main join types at a glance

All four on two tiny tables, showing **every row** returned (no foreign key on purpose, so a student can be in a non-existent club).

```sql
CREATE TABLE clubs    (club_id int PRIMARY KEY, club text);
CREATE TABLE students (id int PRIMARY KEY, name text, club_id int);   -- no FK on purpose
INSERT INTO clubs    VALUES (10, 'Chess'), (20, 'Drama'), (30, 'Robotics');
INSERT INTO students VALUES (1, 'Ana', 10), (2, 'Ben', 20), (3, 'Cai', 20),
                            (4, 'Dev', NULL), (5, 'Eli', 40);
```

Which pairs match on `c.club_id = s.club_id`?

Student	s.club_id	Matching club	Why
Ana	10	Chess	10 = 10
Ben	20	Drama	20 = 20
Cai	20	Drama	20 = 20 (Drama matches **two** students)
Dev	`NULL`	none	`NULL = anything` is UNKNOWN, never TRUE
Eli	40	none	no club 40 (an "orphan" row)
—	—	Robotics (30)	no student has 30

The Venn-style picture

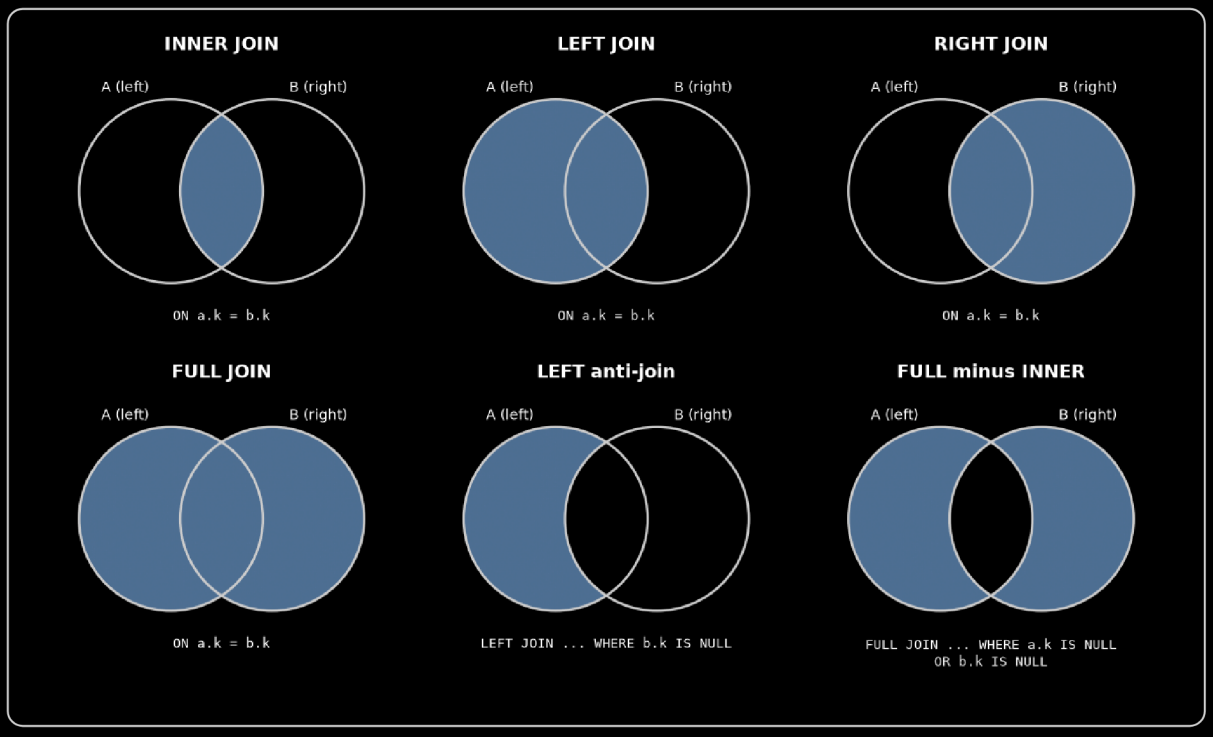


Figure 5.2 — Join "Venn" cheat sheet (drawn by `code/make_join_venn.py`). Left circle = rows of the left table, right circle = rows of the right table, overlap = pairs that satisfy the `ON` condition. Shaded = what the query returns.

Caution: a Venn diagram shows **sets of rows**, but a join produces **pairs** — Drama pairs with Ben *and* Cai, so the "overlap" can exceed either table. Use it to remember *which unmatched rows are kept*; the tables below show *how many rows*.

The result of each join, row by row

`INNER JOIN` — only the matched pairs (3 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s INNER JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club | club
----+------+--------+--------+-------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
(3 rows)
```

`LEFT JOIN` — the matched pairs **plus** every unmatched *left* row, padded with `NULL`s on the right (5 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id;
```

```
 id | name | s_club | c_club |  club
----+------+--------+--------+--------
  1 | Ana  |     10 |     10 | Chess
  2 | Ben  |     20 |     20 | Drama
  3 | Cai  |     20 |     20 | Drama
  4 | Dev  | (null) | (null) | (null)
  5 | Eli  |     40 | (null) | (null)
(5 rows)
```

`RIGHT JOIN` — the matched pairs **plus** every unmatched *right* row, padded with `NULL`s on the left (4 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
ORDER BY c.club_id, s.id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
 (null) | (null) | (null) |     30 | Robotics
(4 rows)
```

`FULL JOIN` — matched pairs, plus unmatched rows from **both** sides (6 rows):

```sql
SELECT s.id, s.name, s.club_id AS s_club, c.club_id AS c_club, c.club
FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
ORDER BY s.id NULLS LAST, c.club_id;
```

```
   id   |  name  | s_club | c_club |   club
--------+--------+--------+--------+----------
      1 | Ana    |     10 |     10 | Chess
      2 | Ben    |     20 |     20 | Drama
      3 | Cai    |     20 |     20 | Drama
      4 | Dev    | (null) | (null) | (null)
      5 | Eli    |     40 | (null) | (null)
 (null) | (null) | (null) |     30 | Robotics
(6 rows)
```

Summary of the same example:

Join	Matched pairs	+ unmatched left (Dev, Eli)	+ unmatched right (Robotics)	Rows
`INNER`	✓			3
`LEFT`	✓	✓		5
`RIGHT`	✓		✓	4
`FULL`	✓	✓	✓	6
`CROSS`	every pair, no condition			5 × 3 = 15

In the `LEFT` result, Dev's `s_club` is `NULL` from his own data; Eli's `c_club` is `NULL` **added by the join**. A right-side `NULL` after a `LEFT JOIN` means "no partner" — unless that column can be `NULL` in the data, so test "no match" on a never-`NULL` right column, typically its primary key.

The edges of the Venn diagram: "only in one side"

`WHERE <right key> IS NULL` after a `LEFT JOIN` keeps only rows with no partner — the left-only crescent (an **anti-join**, §5.8). Likewise for the right side and "only in one of them":

```
SELECT s.name FROM students s LEFT JOIN clubs c ON c.club_id = s.club_id
WHERE c.club_id IS NULL ORDER BY s.id;

SELECT c.club FROM students s RIGHT JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL;

SELECT s.name, c.club FROM students s FULL JOIN clubs c ON c.club_id = s.club_id
WHERE s.id IS NULL OR c.club_id IS NULL ORDER BY s.id NULLS LAST;
```

Output:

```
 name
------
 Dev
 Eli
(2 rows)

   club
----------
 Robotics
(1 row)

  name  |   club
--------+----------
 Dev    | (null)
 Eli    | (null)
 (null) | Robotics
(3 rows)
```

5.3 INNER JOIN

What it is

An **inner join** returns only the pairs for which the join condition is true; rows with no partner disappear. `JOIN` alone means `INNER JOIN`.

How it works

Per left row, find right rows where `ON` is `TRUE` and output one combined row per pair — 3 matches → 3 rows, 0 matches → none. `NULL` keys never match (the comparison is UNKNOWN).

```
SELECT e.emp_id, e.first_name, d.name AS department
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | first_name | department
--------+------------+-------------
      2 | Vikram     | Engineering
      3 | Neha       | Engineering
 ...
     14 | Li         | Support
(13 rows)
```

Asha (1) and Rohan (15) are missing (`dept_id` `NULL`), and so is Research (no employee points to it).

Table aliases (`e`, `d`) are needed when a column exists in both tables (`dept_id`). Always qualify columns — `e.dept_id` — or you get `column reference "dept_id" is ambiguous`.

Filters with joins

For an inner join a condition gives the same result in `ON` or `WHERE`; convention: relationship in `ON`, other filters in `WHERE`. (For outer joins placement **changes the result** — §5.9.)

```
SELECT e.first_name, e.salary, d.name AS department, d.location
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
WHERE d.location = 'Mumbai' AND e.salary > 70000
ORDER BY e.salary DESC;
```

Output:

```
 first_name |  salary   | department | location
------------+-----------+------------+----------
 Daniel     | 150000.00 | Sales      | Mumbai
 Lucas      | 120000.00 | Marketing  | Mumbai
 Priya      |  72000.00 | Sales      | Mumbai
(3 rows)
```

Joining three or more tables

Each `JOIN` adds one table to the result so far. Orders with customer and sales rep:

```
SELECT o.order_id, c.first_name AS customer, e.first_name AS rep, o.status
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN employees e ON e.emp_id = o.sales_rep_id
ORDER BY o.order_id;
```

Output:

```
 order_id | customer |  rep  |  status
----------+----------+-------+-----------
      101 | Anita    | Priya | delivered
      103 | Chen     | Arjun | delivered
      105 | Divya    | Priya | cancelled
      107 | Fatima   | Sara  | shipped
      108 | Ben      | Priya | delivered
      110 | George   | Arjun | shipped
      112 | Ethan    | Sara  | shipped
      113 | Anita    | Priya | pending
(8 rows)
```

Only 8 of 14 orders: the 6 online orders (`sales_rep_id = NULL`) were dropped by the inner join to `employees` (§5.10 keeps them with a `LEFT JOIN`).

A **many-to-many** relationship (one employee on many projects, one project with many employees) goes through a **junction table** (`employee_projects`, each row linking one employee to one project):

```
SELECT p.name AS project, d.name AS dept, e.first_name, ep.role, ep.hours
FROM employee_projects ep
JOIN employees   e ON e.emp_id     = ep.emp_id
JOIN projects    p ON p.project_id = ep.project_id
JOIN departments d ON d.dept_id    = p.dept_id
WHERE p.project_id IN (1, 2)
ORDER BY p.project_id, ep.hours DESC;
```

Output:

```
    project     |    dept     | first_name |   role    | hours
----------------+-------------+------------+-----------+-------
 Checkout Revamp | Engineering | Neha       | Developer |   400
 Checkout Revamp | Engineering | Rahul      | Developer |   350
 Checkout Revamp | Engineering | Vikram     | Lead      |   120
 Mobile App      | Engineering | Neha       | Lead      |   300
 Mobile App      | Engineering | Meera      | Developer |   280
 Mobile App      | Engineering | Rohan      | Tester    |    90
(6 rows)
```

Join first, then group (Chapter 4):

```sql
SELECT e.first_name AS rep, count(DISTINCT o.order_id) AS orders,
       round(sum(oi.quantity * oi.unit_price * (1 - oi.discount)), 2) AS revenue
FROM orders o
JOIN employees e    ON e.emp_id = o.sales_rep_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.status <> 'cancelled'
GROUP BY e.emp_id, e.first_name
ORDER BY revenue DESC;
```

Output:

```
  rep  | orders | revenue
-------+--------+---------
 Priya |      3 | 4141.04
 Sara  |      2 |  879.91
 Arjun |      2 |  687.00
(3 rows)
```

`count(DISTINCT o.order_id)`, not `count(*)`: after joining `order_items` each order appears once *per line* (fan-out, §5.9).

Non-equi joins

The condition can be any boolean expression — order lines paid below the current list price:

```sql
SELECT oi.order_id, p.name, p.unit_price AS list_price, oi.unit_price AS paid
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
               AND oi.unit_price < p.unit_price
ORDER BY oi.order_id;
```

Output:

```
 order_id |     name      | list_price |  paid
----------+---------------+------------+---------
      104 | Standing Desk |     499.00 |  479.00
      108 | Laptop Pro 14 |    1299.00 | 1249.00
(2 rows)
```

Also common: value-to-range (`ON s.amount BETWEEN b.low AND b.high` for tax brackets, grade bands) and time windows (`ON b.t > a.t AND b.t <= a.t + interval '1 day'`).

In the interview

> **30-second answer.** "An inner join returns the combinations of rows from both tables that satisfy the join condition; unmatched rows on either side are dropped, and `NULL` keys never match. If a row has several matches it appears several times."

- *Q: Is `JOIN` the same as `INNER JOIN`?* — Yes.
- *Q: `ON` vs `WHERE` for an inner join?* — Same result; put the relationship in `ON` for readability. For outer joins they differ.

- *Q: Why did my row count go up after a join?* — One-to-many (one left row relates to many right rows): each left row repeats once per matching right row.

Self-check

1. Can `employees JOIN departments` return more than 15 rows? → No: each employee has at most one department (the key is the department's primary key). Joining to a many-side (`orders JOIN order_items`) can.

5.4 LEFT OUTER JOIN

What it is

A **left outer join** (`LEFT JOIN`; `OUTER` is optional) returns **every row of the left table**: matched rows as in an inner join, unmatched rows once with every right column `NULL` (**NULL padding** / **NULL extension**) — like a class register listing every student, with a blank mark for those who missed the exam.

Why it matters

It answers what inner joins silently get wrong: "**all** customers with order count — **including zero**", "all employees, even unassigned", "every product and its sales".

How it works (rows of employees LEFT JOIN departments)

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 ...                                (13 matched rows)
 Li         |      5 |      5 | Support
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(15 rows)
```

The 13 matched employees appear as in the inner join; Asha and Rohan appear **once** each, `NULL`-padded; Research is absent (only unmatched **left** rows are kept).

Rule of thumb: `LEFT JOIN` **never returns fewer rows than the left table has.** It returns more if some left rows match several right rows.

The everyday use: counts that include zero

```
SELECT c.customer_id, c.first_name, count(o.order_id) AS orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name
ORDER BY orders, c.customer_id;
```

Output:

```
 customer_id | first_name | orders
-------------+------------+--------
           9 | Hana       |      0
          10 | Ivan       |      0
           4 | Divya      |      1
 ...
           1 | Anita      |      3
(10 rows)
```

It must be `count(o.order_id)` (a right-table column), not `count(*)`, which counts the padded row as 1 (§5.9).

When not to use it

- Only **left rows that have a match**, no right columns? A semi-join with `EXISTS` is clearer and never duplicates rows (§5.8).
- Match required? Use `INNER JOIN` — a `LEFT JOIN` plus `WHERE` on a right column silently becomes one anyway (§5.9).

Common mistakes

- Filtering the right table in `WHERE` (back to an inner join) — put it in `ON`.
- `a LEFT JOIN b … JOIN c ON c.x = b.x`: the inner join drops the padded rows again (`b.x` is `NULL`). Make it a `LEFT JOIN` too.
- `count(*)` for "how many matches".

In the interview

> **30-second answer.** "A left join returns every row of the left table; where there is a matching right row the columns are filled in, otherwise they're `NULL`. It's the standard way to keep rows with no related data — like customers with zero orders — and, with `WHERE right.pk IS NULL`, to find rows with no match."

- *Q: How many rows can a left join return?* — At least as many as the left table; more if there are one-to-many matches.
- *Q: Find employees without a department.* — `LEFT JOIN departments … WHERE d.dept_id IS NULL`, or simply `WHERE dept_id IS NULL` on employees — plus rows whose `dept_id` points to a missing department, if there is no foreign key.

Self-check

1. Would swapping the tables change the result? → Yes, it becomes "all departments" (§5.5).

5.5 RIGHT OUTER JOIN

What it is

A **right outer join** mirrors it: **every row of the right table**, left side `NULL`-padded. `A RIGHT JOIN B` returns the same rows as `B LEFT JOIN A` (only column order differs).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 ...                                (13 matched rows)
 Li         |      5 |      5 | Support
 (null)     | (null) |      6 | Research
(14 rows)
```

Research appears with `NULL` employee columns; Asha and Rohan (unmatched *left* rows) are gone. `departments d LEFT JOIN employees e` gives the same 14 rows.

When to use it

Rarely. Most teams write every outer join as `LEFT JOIN` with the "keep everything" table first — easier to read. `RIGHT JOIN` helps when appending a table to a long join chain and keeping all *its* rows without restructuring. SQLite supports `RIGHT` and `FULL` joins only from 3.39.0 (2022).

Self-check

1. Rewrite `orders o RIGHT JOIN customers c ON …` as a left join. → `customers c LEFT JOIN orders o ON …`.

5.6 FULL OUTER JOIN

What it is

A **full outer join** keeps **all rows from both tables**, padding whichever side is unmatched — the union of the left- and right-join results (matched pairs once).

```
SELECT e.first_name, e.dept_id AS e_dept, d.dept_id AS d_dept, d.name
FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id
ORDER BY d.dept_id NULLS LAST, e.emp_id;
```

Output:

```
 first_name | e_dept | d_dept |    name
------------+--------+--------+-------------
 Vikram     |      1 |      1 | Engineering
 ...                                (13 matched rows)
 (null)     | (null) |      6 | Research
 Asha       | (null) | (null) | (null)
 Rohan      | (null) | (null) | (null)
(16 rows)
```

Row counts line up with the edge cases — 13 matched pairs, 2 unmatched employees, 1 unmatched department:

```
SELECT (SELECT count(*) FROM employees e JOIN departments d ON d.dept_id = e.dept_id)       AS inner_rows,
       (SELECT count(*) FROM employees e LEFT JOIN departments d ON d.dept_id = e.dept_id)  AS left_rows,
       (SELECT count(*) FROM employees e RIGHT JOIN departments d ON d.dept_id = e.dept_id) AS right_rows,
       (SELECT count(*) FROM employees e FULL JOIN departments d ON d.dept_id = e.dept_id)  AS full_rows;
```

```
 inner_rows | left_rows | right_rows | full_rows
------------+-----------+------------+-----------
         13 |        15 |         14 |        16
(1 row)
```

In general: `full = inner + unmatched left + unmatched right`, `left = inner + unmatched left`, `right = inner + unmatched right`.

Real-world use: reconciliation

`FULL JOIN` shines in **reconciliation** — checking two records of the same facts agree, where each side may have entries the other lacks: planned vs actual, yesterday's vs today's snapshot, bank statement vs ledger.

```sql
CREATE TABLE planned (emp_id int, hours int);
CREATE TABLE actual  (emp_id int, hours int);
INSERT INTO planned VALUES (3, 300), (4, 200), (5, 250);
INSERT INTO actual  VALUES (3, 310), (5, 250), (7, 40);
SELECT coalesce(p.emp_id, a.emp_id) AS emp_id, p.hours AS planned, a.hours AS actual,
       CASE WHEN p.emp_id IS NULL THEN 'unplanned work'
            WHEN a.emp_id IS NULL THEN 'no work logged'
            WHEN p.hours <> a.hours THEN 'different'
            ELSE 'ok' END AS verdict
FROM planned p FULL JOIN actual a ON a.emp_id = p.emp_id
ORDER BY 1;
```

Output:

```
 emp_id | planned | actual |    verdict
--------+---------+--------+---------------
      3 |     300 |    310 | different
      4 |     200 | (null) | no work logged
      5 |     250 |    250 | ok
      7 |  (null) |     40 | unplanned work
(4 rows)
```

`COALESCE(p.emp_id, a.emp_id)` picks whichever key is present (one is `NULL` on unmatched rows).

Dialects

Database	FULL OUTER JOIN
PostgreSQL, SQL Server, Oracle	supported
SQLite	supported since 3.39.0
MySQL / MariaDB	**not supported** — emulate: `SELECT … FROM a LEFT JOIN b ON … UNION SELECT … FROM a RIGHT JOIN b ON …` (or `UNION ALL` with the right-join half filtered to `WHERE a.key IS NULL` to avoid removing genuine duplicates)

PostgreSQL restricts `FULL JOIN` conditions to ones runnable as a merge or hash join (§5.10) — in practice at least one equality between the sides; `ON a.x < b.y` alone is an error.

In the interview

> **30-second answer.** "A full outer join returns every row from both tables, matching where it can and padding the other side with `NULL`s where it can't. It's used to reconcile two datasets. MySQL lacks it, so you emulate it with a `LEFT JOIN` `UNION` a `RIGHT JOIN`."

- *Q: How do you find rows that are in exactly one of two tables?* — `FULL JOIN … WHERE a.pk IS NULL OR b.pk IS NULL` (or two anti-joins, or `EXCEPT` — "rows in the first result but not the second" — both ways; Chapter 6).

5.7 SELF JOIN

What it is

A **self join** joins a table **to itself** — an ordinary join with the same table twice under two aliases ("employee" row vs "manager" row). Use it when rows refer to rows of the **same** table (hierarchy via `manager_id`) or to compare rows with each other (pairs, sequences).

How it works

```
SELECT e.emp_id, e.first_name AS employee, m.first_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id
ORDER BY e.emp_id;
```

Output:

```
 emp_id | employee | manager
--------+----------+---------
      1 | Asha     | (null)
      2 | Vikram   | Asha
      3 | Neha     | Vikram
 ...
     15 | Rohan    | Vikram
(15 rows)
```

`e` is the employee copy, `m` the manager copy; `m.emp_id = e.manager_id` pairs each employee with their manager's row. `LEFT JOIN` keeps Asha (`manager_id` `NULL`); an inner join would drop her.

More self-join patterns

Compare each employee with their manager:

```
SELECT e.first_name, e.salary, m.first_name AS manager, m.salary AS mgr_salary
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary >= 0.7 * m.salary
ORDER BY e.emp_id;
```

```
 first_name |  salary   | manager | mgr_salary
------------+-----------+---------+------------
 Vikram     | 185000.00 | Asha    |  250000.00
 Neha       | 142000.00 | Vikram  |  185000.00
(2 rows)
```

Count direct reports (join, then group by the manager):

```
SELECT m.first_name AS manager, count(*) AS direct_reports
FROM employees e JOIN employees m ON m.emp_id = e.manager_id
GROUP BY m.emp_id, m.first_name
ORDER BY direct_reports DESC, manager;
```

```
 manager | direct_reports
---------+----------------
 Vikram  |              5
 Asha    |              4
 Daniel  |              3
 Kiran   |              1
 Lucas   |              1
(5 rows)
```

Pairs sharing a value — `a.emp_id < b.emp_id` removes self-pairs (Arjun–Arjun) and mirror duplicates (Arjun–Sara *and* Sara–Arjun):

```
SELECT a.first_name AS emp_a, b.first_name AS emp_b, a.salary
FROM employees a JOIN employees b ON a.salary = b.salary AND a.emp_id < b.emp_id
ORDER BY a.salary;
```

```
 emp_a | emp_b |  salary
-------+-------+----------
 Arjun | Sara  | 68000.00
 Rahul | Meera | 98000.00
(2 rows)
```

Two levels up with a second self join (arbitrary depth needs a recursive CTE — a Common Table Expression is a named subquery written with `WITH`; a *recursive* one keeps joining to its own previous output until nothing new appears; Chapter 7):

```
SELECT e.first_name AS employee, m.first_name AS manager, g.first_name AS grand_manager
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
LEFT JOIN employees g ON g.emp_id = m.manager_id
WHERE e.dept_id = 1
ORDER BY e.emp_id;
```

```
 employee | manager | grand_manager
----------+---------+--------------
 Vikram   | Asha    | (null)
 Neha     | Vikram  | Asha
 Rahul    | Vikram  | Asha
 Meera    | Vikram  | Asha
(4 rows)
```

Rows **in time** — customers reordering within 60 days:

```
SELECT a.customer_id, a.order_id AS first_order, b.order_id AS next_order,
       b.placed_at::date - a.placed_at::date AS days_between
FROM orders a JOIN orders b
  ON b.customer_id = a.customer_id AND b.placed_at > a.placed_at
 AND b.placed_at <= a.placed_at + interval '60 days'
ORDER BY a.customer_id, a.order_id;
```

```
 customer_id | first_order | next_order | days_between
-------------+-------------+------------+--------------
           1 |         101 |        104 |           39
(1 row)
```

"Compare with the previous row" problems (consecutive days, streaks) are solved like this or, more efficiently, with `LAG` / `LEAD` window functions (read the previous/next row's value without collapsing rows; Chapter 8).

Common mistakes

- Forgetting aliases (`employees JOIN employees` is an error: `table name "employees" specified more than once`).
- Using an inner join for hierarchies and losing the root (the CEO).
- Getting every pair twice (and each row paired with itself) — use `a.id < b.id`.

In the interview

> **30-second answer.** "A self join joins a table to itself using two aliases — for hierarchies like employee → manager, for finding pairs of rows with something in common, and for comparing a row with a related row of the same table. Use a left join to keep rows without a parent, and `a.id < b.id` to avoid duplicate pairs."

- *Q: Employees who earn more than their manager?* — `FROM employees e JOIN employees m ON m.emp_id = e.manager_id WHERE e.salary > m.salary`. (In our data: nobody.)
- *Q: How do you get the whole chain of managers?* — A recursive CTE; self joins only reach a fixed depth.

5.8 Semi-joins and anti-joins

What they are

- **Semi-join** — "rows of A that **have at least one** match in B" (customers who ordered); each A row **at most once**, no B columns needed.
- **Anti-join** — "rows of A with **no** match in B" (customers who never ordered).

There is no `SEMI JOIN` keyword: write `EXISTS` / `IN` (semi) and `NOT EXISTS` / `LEFT JOIN … IS NULL` / `NOT IN` (anti). The planner (the component deciding how to execute a query) recognises these shapes and uses dedicated *semi-join / anti-join* algorithms.

Semi-join: EXISTS vs a plain JOIN

```
SELECT c.customer_id, c.first_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

```
 customer_id | first_name
-------------+------------
           1 | Anita
           2 | Ben
           3 | Chen
           4 | Divya
           5 | Ethan
           6 | Fatima
           7 | George
           8 | Anita
(8 rows)
```

A plain `customers JOIN orders` answers a different question — one row **per order** (14 rows, Anita three times). `SELECT DISTINCT` fixes it but builds all pairs then de-duplicates and hides the intent; `EXISTS` stops at each customer's first match.

Anti-join: three ways

```
-- 1. NOT EXISTS
SELECT c.customer_id, c.first_name FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
ORDER BY 1;

-- 2. LEFT JOIN ... IS NULL
SELECT c.customer_id, c.first_name FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL
ORDER BY 1;

-- 3. NOT IN
SELECT c.customer_id, c.first_name FROM customers c
WHERE c.customer_id NOT IN (SELECT customer_id FROM orders)
ORDER BY 1;
```

Output (identical for all three here):

```
 customer_id | first_name
-------------+------------
           9 | Hana
          10 | Ivan
(2 rows)
```

Form	NULL-safe?	Notes
`NOT EXISTS (correlated subquery)` (a subquery that refers to the outer row)	**Yes**	Clearest intent; PostgreSQL plans it as an anti join. **Recommended.**
`LEFT JOIN … WHERE b.pk IS NULL`	Yes, if you test a never-NULL column (the PK)	Also planned as an anti join; handy when you want other columns from the join.
`NOT IN (subquery)`	**No**	If the subquery returns even one `NULL`, the result is **empty** — see Chapter 6. Also often planned less efficiently.

`EXPLAIN` (shows the chosen strategy; `Seq Scan` = read the whole table) runs `NOT EXISTS` as a **Hash Anti Join**:

```
EXPLAIN (COSTS OFF)
SELECT c.customer_id FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

```
                  QUERY PLAN
-----------------------------------------------
 Hash Anti Join
   Hash Cond: (c.customer_id = o.customer_id)
   ->  Seq Scan on customers c
   ->  Hash
         ->  Seq Scan on orders o
(5 rows)
```

More anti-joins — the product never sold and the department with no staff:

```
SELECT p.name FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);

SELECT d.name FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;
```

```
   name
-----------
 Desk Lamp
(1 row)

   name
----------
 Research
(1 row)
```

In the interview

30-second answer. "A semi-join returns rows of A that have a match in B, once each — write it with `EXISTS` or `IN`. An anti-join returns rows of A with no match — write it with `NOT EXISTS` or `LEFT JOIN … WHERE b.pk IS NULL`. Avoid `NOT IN` with a subquery on a nullable column: one `NULL` makes it return nothing."

- *Q: Customers who never placed an order?* — Any of the three forms; say why you prefer `NOT EXISTS`.
- *Q: Why not `JOIN` + `DISTINCT` for a semi-join?* — It multiplies then de-duplicates; `EXISTS` stops at the first match and states the intent.

Self-check

1. In `LEFT JOIN … IS NULL`, why test `o.order_id`, not `o.sales_rep_id`? → `sales_rep_id` is `NULL` on real rows (online orders); a `NULL` primary key can only be padding.

5.9 Join pitfalls

Pitfall 1 — fan-out (row multiplication)

Joining a parent to **two child tables** at once pairs every child of one kind with every child of the other: Engineering's 4 employees × 2 projects = 8 rows, every salary counted twice:

```
SELECT d.name, count(*) AS joined_rows, sum(e.salary) AS wrong_payroll
FROM departments d
JOIN employees e ON e.dept_id = d.dept_id
JOIN projects  p ON p.dept_id = d.dept_id
WHERE d.dept_id = 1
GROUP BY d.name;
```

```
    name     | joined_rows | wrong_payroll
-------------+-------------+---------------
 Engineering |           8 |    1046000.00
(1 row)
```

The real payroll is 523,000. **Fix:** aggregate each child separately (derived table — a subquery in `FROM` — or CTE) to one row per department, *then* join:

```
SELECT d.name, e.payroll, p.projects
FROM departments d
JOIN (SELECT dept_id, sum(salary) AS payroll FROM employees GROUP BY dept_id) e ON e.dept_id = d.dept_id
JOIN (SELECT dept_id, count(*) AS projects FROM projects GROUP BY dept_id)     p ON p.dept_id = d.dept_id
ORDER BY d.dept_id;
```

```
    name     |  payroll  | projects
-------------+-----------+----------
 Engineering | 523000.00 |        2
 Marketing   | 181000.00 |        1
 Finance     | 211000.00 |        1
 Support     |  64000.00 |        1
(4 rows)
```

Signs: totals that are exact multiples of the right answer; `count(*)` above the number of parents. `count(DISTINCT …)` hides it for counts but **not** sums.

Pitfall 2 — a WHERE on the right table undoes a LEFT JOIN

Goal: every customer with their number of **delivered** orders, 0 if none.

```
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- wrong: customers with 0 vanish
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           5 |         1
           8 |         1
(5 rows)
```

Only 5: `WHERE o.status = 'delivered'` ran **after** the join (logical order, Chapter 4) and discarded the padded rows (Hana, Ivan: `o.status = NULL`) and customers with no delivered order. Put the right-table condition **in** `ON`, so it limits which rows *match*, not which *survive*:

```
SELECT c.customer_id, count(o.order_id) AS delivered
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id
                                    AND o.status = 'delivered'
GROUP BY c.customer_id ORDER BY c.customer_id;          -- right: condition in ON
```

```
 customer_id | delivered
-------------+-----------
           1 |         2
           2 |         2
           3 |         1
           4 |         0
 ...
          10 |         0
(10 rows)
```

Condition on the right table of a LEFT JOIN placed in…	Effect
`ON`	Decides which right rows count as a match; unmatched left rows are still kept (padded).
`WHERE`	Filters the joined result; padded rows have `NULL` there and are removed → behaves like an `INNER JOIN`.

(A `WHERE right.pk IS NULL` test is the deliberate exception — that's the anti-join.) Conditions on the **left** table belong in `WHERE`; put in `ON` they don't remove left rows, they just stop them matching.

Pitfall 3 — NULL keys never match

```
CREATE TABLE a (k int, label text);
CREATE TABLE b (k int, label text);
INSERT INTO a VALUES (1, 'a1'), (2, 'a2'), (NULL, 'a-null');
INSERT INTO b VALUES (1, 'b1'), (3, 'b3'), (NULL, 'b-null');
SELECT a.label, b.label FROM a JOIN b ON a.k = b.k;
SELECT a.label, b.label FROM a JOIN b ON a.k IS NOT DISTINCT FROM b.k ORDER BY 1;
```

```
 label | label
-------+-------
 a1    | b1
(1 row)

 label  | label
--------+--------
 a-null | b-null
 a1     | b1
(2 rows)
```

`NULL = NULL` is UNKNOWN, so `NULL` keys don't pair. To match `NULL` with `NULL` (e.g. comparing snapshots of a nullable column) use `IS NOT DISTINCT FROM` — but PostgreSQL can't run it as a hash or merge join, so it can be slow on large tables.

Pitfall 4 — COUNT(*) after an outer join

```
SELECT d.name, count(*) AS count_star, count(e.emp_id) AS count_col
FROM departments d LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE d.dept_id IN (5, 6)
GROUP BY d.name ORDER BY d.name;
```

```
   name   | count_star | count_col
----------+------------+-----------
 Research |          1 |         0
 Support  |          1 |         1
(2 rows)
```

Research has no employees but `count(*)` says 1 (the padded row). Count a right-side, never-NULL column.

Other pitfalls, briefly

- **Wrong columns** — `ON e.emp_id = d.dept_id` "works" (both integers) and returns nonsense; foreign keys document the right paths.
- **Type mismatches** — `text` to `integer` fails in PostgreSQL; casting in the condition (`ON a.code::int = b.id`) prevents index use on `a.code` (an **index** is a sorted lookup structure for finding rows by value without reading everything).
- **Duplicated keys in a lookup table** — if the "one" side isn't unique, every match fans out. Check with `GROUP BY key HAVING count(*) > 1`.

Self-check

1. A query sums order amounts and joins both `order_items` and `payments` to `orders`. What goes wrong? → Fan-out: each item pairs with each payment, inflating the sums. Aggregate each child first.
2. `a LEFT JOIN b ON a.id = b.a_id WHERE b.active` — what kind of join is this really? → An inner join (padded rows have `b.active = NULL`). Move `b.active` to `ON`.

5.10 USING, NATURAL JOIN, multi-table joins, and how joins are executed

USING

When join columns share a **name**, `USING (col)` is shorthand for `ON a.col = b.col`; the column appears **once** in the output (and `SELECT *`) and is referenced without a prefix:

```
SELECT order_id, customer_id, first_name, status
FROM orders JOIN customers USING (customer_id)
WHERE order_id <= 103
ORDER BY order_id;
```

```
 order_id | customer_id | first_name |  status
----------+-------------+------------+-----------
      101 |           1 | Anita      | delivered
      102 |           2 | Ben        | delivered
      103 |           3 | Chen       | delivered
(3 rows)
```

In `FULL JOIN … USING` the merged column is `COALESCE(left.col, right.col)`, filled for unmatched rows on either side:

```
SELECT dept_id, first_name, name
FROM employees FULL JOIN departments USING (dept_id)
WHERE first_name IS NULL OR name IS NULL
ORDER BY dept_id NULLS LAST;
```

```
 dept_id | first_name |   name
---------+------------+----------
       6 | (null)     | Research
  (null) | Asha       | (null)
  (null) | Rohan      | (null)
(3 rows)
```

NATURAL JOIN — and why to avoid it

`NATURAL JOIN` joins on **every** same-named column pair without listing them — convenient until a table gains a column:

```
SELECT count(*) AS natural_rows FROM projects NATURAL JOIN departments;
SELECT count(*) AS using_rows   FROM projects JOIN departments USING (dept_id);
```

```
 natural_rows
--------------
            0
(1 row)

 using_rows
------------
          5
(1 row)
```

`projects` and `departments` share `dept_id` — and `name` and `budget`; all three had to match, so nothing did. Schemas (table definitions) change (someone adds `created_at` to both) and `NATURAL JOIN` silently changes meaning. **Name your join columns.**

	ON	USING	NATURAL
Columns compared	any expression	listed same-named columns	*all* same-named columns
Duplicated key column in `SELECT *`	yes (both copies)	no (one merged column)	no
Safe against schema changes	yes	yes	**no**
SQL Server support	yes	no	no

Multi-table joins

Every order line with customer, product and sales rep, keeping online orders:

```
SELECT o.order_id, c.first_name AS customer, p.name AS product, oi.quantity,
       coalesce(e.first_name, 'online') AS rep
FROM orders o
JOIN customers c    ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products p     ON p.product_id  = oi.product_id
LEFT JOIN employees e ON e.emp_id    = o.sales_rep_id
WHERE o.order_id BETWEEN 101 AND 104
ORDER BY o.order_id, p.name;
```

```
 order_id | customer |           product           | quantity |  rep
----------+----------+-----------------------------+----------+--------
      101 | Anita    | Laptop Pro 14               |        1 | Priya
      101 | Anita    | Wireless Mouse              |        1 | Priya
      102 | Ben      | Noise-Cancelling Headphones |        2 | online
      103 | Chen     | Mechanical Keyboard         |        1 | Arjun
      103 | Chen     | USB-C Hub                   |        2 | Arjun
      104 | Anita    | Ergonomic Chair             |        1 | online
      104 | Anita    | Standing Desk               |        1 | online
(7 rows)
```

A department overview from outer joins, so Sales (no projects) and Research (nothing) still appear:

```sql
SELECT d.name AS department,
       count(DISTINCT e.emp_id)     AS employees,
       count(DISTINCT p.project_id) AS projects,
       coalesce(sum(ep.hours), 0)   AS hours_on_own_projects
FROM departments d
LEFT JOIN employees e          ON e.dept_id = d.dept_id
LEFT JOIN projects p           ON p.dept_id = d.dept_id
LEFT JOIN employee_projects ep ON ep.project_id = p.project_id AND ep.emp_id = e.emp_id
GROUP BY d.dept_id, d.name
ORDER BY d.dept_id;
```

```
 department  | employees | projects | hours_on_own_projects
-------------+-----------+----------+----------------------
 Engineering |         4 |        2 |                  1450
 Sales       |         4 |        0 |                     0
 Marketing   |         2 |        1 |                   350
 Finance     |         2 |        1 |                   240
 Support     |         1 |        1 |                    60
 Research    |         0 |        0 |                     0
(6 rows)
```

It deliberately fans out (employees × projects) and is correct only because counts use `DISTINCT` and hours come from `employee_projects` joined on **both** keys (each assignment once). When in doubt, pre-aggregate (§5.9).

Rules: (1) inner joins can be written in any order — same result, the planner picks the physical order; (2) **outer joins are not freely reorderable** — `(a LEFT JOIN b) JOIN c` can differ from `a LEFT JOIN (b JOIN c)`, so write the order you mean with the "keep everything" table first; (3) every table after the first needs its own join condition, or you get a Cartesian product.

How databases execute joins

Joins don't run as filtered Cartesian products. The planner picks one of three algorithms per join from table sizes, indexes and statistics (stored summaries of row counts and value spread) — visible in `EXPLAIN` (Chapter 12):

Algorithm	How it works	Good when	Cost (roughly)
Nested loop	For each outer row, look up matching inner rows (ideally through an index)	Outer side small, index on the inner join key; also the only option for non-equi joins	*outer rows × cost of one lookup*
Hash join	Build a hash table (a lookup structure that finds a key in about one step) on the smaller input's key, then probe it with each row of the other	Equi-joins of large unsorted inputs	*O(n + m)* (work grows with the total size of both inputs), needs memory for the hash table
Merge join	Sort both inputs by the key (or read them in index order), then walk them together	Inputs already sorted / indexed; very large inputs	*O(n + m)* after sorting

Index your foreign-key columns (`orders.customer_id`, `order_items.product_id` …): PostgreSQL indexes primary keys automatically but **not** foreign keys, and parent-to-child nested loops depend on them.

Dialect summary for joins

Feature	PostgreSQL	MySQL 8	SQLite	SQL Server
`INNER`, `LEFT`, `CROSS`	yes	yes	yes	yes
`RIGHT JOIN`	yes	yes	3.39+	yes
`FULL JOIN`	yes	**no** (emulate with `UNION`)	3.39+	yes
`USING`, `NATURAL`	yes	yes	yes	**no**
`LATERAL` (Chapter 6)	yes	8.0.14+	no	`CROSS APPLY` / `OUTER APPLY`
`IS NOT DISTINCT FROM`	yes	`<=>`	`IS`	2022+

In the interview

30-second answer (whole chapter). "A join pairs rows from two tables that satisfy a condition — conceptually a filtered Cartesian product. Inner join keeps matches only; left/right keep all rows of one side with `NULL` padding; full keeps both; cross gives every combination. Self joins use two aliases on one table. For 'has a match / has no match' use `EXISTS` / `NOT EXISTS`. Watch for fan-out, filters on the outer side in `WHERE`, and `NULL` keys."

- *Q: What's the difference between `WHERE` and `ON` in a left join?* — `ON` decides matches; `WHERE` filters the result and removes padded rows.
- *Q: What join algorithms do you know?* — Nested loop, hash, merge; the optimizer chooses based on sizes, indexes and sort order.
- *Q: How do you do a full outer join in MySQL?* — `LEFT JOIN` `UNION` `RIGHT JOIN`.

Self-check

1. What columns does `SELECT *` return for `orders JOIN customers USING (customer_id)`? → `customer_id` once, then the other columns of `orders` and `customers`.
2. Why is `NATURAL JOIN` risky? → It joins on every same-named column, which changes when schemas change.
3. Which physical join works for `ON a.x < b.y`? → Nested loop (hash and merge need equality).

Chapter 5 TL;DR

Join	Returns	Typical use
`CROSS JOIN`	every pair (m × n)	grids, combinations
`INNER JOIN`	matched pairs only	"orders with their customers"
`LEFT JOIN`	all left rows + matches, `NULL`-padded	"all customers, with orders if any"
`RIGHT JOIN`	all right rows + matches	same, mirrored — write as `LEFT`
`FULL JOIN`	all rows of both sides	reconciliation
Self join	table joined to itself (two aliases)	hierarchies, pairs, previous/next
Semi-join (`EXISTS`)	left rows with ≥ 1 match, once each	"customers who ordered"
Anti-join (`NOT EXISTS`, `LEFT … IS NULL`)	left rows with no match	"customers who never ordered"

Pitfalls: fan-out, right-table filters in `WHERE` after a `LEFT JOIN`, `NULL` keys, `count(*)` on padded rows, `NOT IN` with `NULL`s, `NATURAL JOIN`.

Next: Chapter 6 — Subqueries and Set Operations.